SAVEETHA UNIVERSITY (SIMATS)

B.Tech – Computer Science and Engineering

DATA STRUCTURES AND ALGORITHMS

Course Code: DSA0113

ASSIGNMENT 1 – CO1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Student Name	Nirmal Kumar T
Register Number	1972260020
Course	B.Tech CSE
Course Code	DSA0113
GitHub Repository	https://github.com/Nirmal637/DSA01-1972260020-Assignment1
Completed Section	Task 1 – Understand the Problem and Identify the Objects

Black-and-white academic report

TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY THE OBJECTS

Marks: 4 **Bloom's Level:** K5 – Evaluate

1. Problem understanding

The metro network has three zones: Inner, Middle and Outer. The base fare depends on the farthest zone travelled: Rs 10, Rs 20 or Rs 30 respectively. A peak-hour surcharge of 20% applies during 8:00–10:00 AM and 5:00–8:00 PM. Passengers pay using stored-value smart cards. A journey is allowed only if the balance remaining after fare deduction is at least Rs 50. Students and senior citizens also receive a concession.

The system must calculate fares consistently, prevent a card from falling below the required minimum balance, and keep fare rules maintainable. The following analysis separates the responsibilities of the main objects.

2. Objects, their data and behaviour

Object	Possible data (attributes)	Behaviour (operations)	Justification
SmartCard	Card ID; private stored balance in paise; card status.	checkBalance(); canTravel(fare); deductFare(fare); recharge(amount).	The card owns stored value. Private balance prevents unrelated code from changing it directly or creating an invalid balance.
Passenger	Passenger ID; category (regular, student or senior citizen); associated card.	identifyCategory(); requestJourney(); provide concession eligibility.	Passenger identity and eligibility are separate from the card's money, so concession logic need not be mixed with balance management.
Journey	Origin; destination; farthest zone; entry time; peak status; fare; journey status.	determineZone(); checkPeakPeriod(); calculateFare(); validate(); completeJourney().	Represents one travel transaction and records its route, time, calculated fare and approval/denial result.
Zone	Zone identity: Inner, Middle or Outer; optionally a defined rank.	Represent a valid zone and support determining the farthest zone travelled.	A domain concept used by journeys and fare calculation. C++ enum class can represent the allowed values safely.
FareTable	Base fares; peak surcharge percentage; minimum remaining balance; concession policy reference.	getBaseFare(zone); applyPeakRule(fare); applyConcession(fare, passenger); provideFareRules().	Centralises fare values and policy so revisions do not require editing scattered literals.

Design note: money should be represented internally as integer paise in a later implementation, rather than binary floating-point rupees, to avoid fractional-paisa rounding. Task 1 itself asks for object analysis, not a completed program.

3. Procedural programming compared with OOP

Procedural style	Object-oriented style
Stores card balances, passenger categories, journey details and fare values in separate variables, arrays or structures. Free functions such as calculateFare(), canTravel() and deductFare() operate on those values. Some state may be shared through global variables.	Groups related data and operations into classes such as SmartCard, Passenger, Journey and FareTable. Each object manages the state and operations for the concept it represents.
The programmer must pass the correct balance, passenger type, zone and time to the correct functions. A function may accidentally change data that should be protected.	The class interface controls data access. SmartCard can expose canTravel() and deductFare() while keeping its balance private.
When a fare rule changes, every function or section using that rule must be located. Duplicated fare values can become inconsistent.	A central FareTable or fare-policy component can hold the rule, reducing duplicated logic and the number of places needing modification.

Procedural sequence: read passenger, card and journey data → determine farthest zone → obtain base fare → apply peak and eligible concession rules → check remaining balance → deduct only if permitted. In OOP, Journey coordinates the same work by requesting a fare from FareTable, asking SmartCard whether travel is affordable, and asking SmartCard to deduct only after validation succeeds.

4. OOP principles in the metro system

Encapsulation: SmartCard keeps its balance private. The balance changes only through controlled operations such as recharge() and deductFare(). deductFare() rejects an amount that would leave less than Rs 50. This protects the card invariant.

Abstraction: The gate can call a high-level operation such as Journey.completeJourney() without knowing every internal step of fare calculation, peak checking and balance validation. The public interface exposes what an object does while hiding implementation details.

Inheritance: A Passenger base class could contain common passenger details. StudentPassenger and SeniorCitizenPassenger could derive from it to represent concession-eligible categories. This is useful if they need distinct data or behaviour; otherwise a category field and policy lookup may be simpler.

Polymorphism: If concession calculation is a virtual operation in the Passenger hierarchy, the same call such as passenger.getConcessionPolicy() can produce a student or senior-citizen policy according to the actual passenger type. The fare code can use a common interface.

5. Rules that every correct journey must satisfy

• **Zone-based base fare:** use the farthest zone reached: Inner Rs 10, Middle Rs 20, Outer Rs 30.

• **Peak-hour boundaries:** apply surcharge from 8:00 AM inclusive to 10:00 AM exclusive, and from 5:00 PM inclusive to 8:00 PM exclusive. Therefore 8:00 AM and 5:00 PM are peak; 10:00 AM and 8:00 PM are not.

• **Peak surcharge:** during peak time, add 20% of the applicable base fare.

• **Minimum remaining balance:** allow travel only when balance minus final fare is greater than or equal to Rs 50. A balance exactly equal to fare plus Rs 50 is sufficient.

• **No invalid deduction:** if the balance test fails, deny the journey and leave the balance unchanged. A journey must not make the balance negative or lower than Rs 50.

• **Concession eligibility:** apply a concession only to an eligible student or senior citizen. The scenario does not provide concession percentages or the order of applying concession and peak surcharge; do not guess these policy details.

• **Single fare source:** maintain one authoritative fare table and policy so identical zone and time inputs produce consistent results.

• **Valid journey data:** validate zone, route and time before calculating or deducting a fare. Invalid input must not change the card balance.

6. Alternative concept: purely procedural design

A procedural implementation could store card balances in an array, passenger categories in another array, and journey details in variables or structures. Global constants could hold the base fares, peak surcharge and minimum balance. Free functions could determine the zone, check peak time, calculate fare, test travel eligibility and update the balance.

Illustrative procedural pseudocode

```
CONSTANT MIN_BALANCE = 5000 paise
CONSTANT PEAK_RATE = 20 percent
CONSTANT BASE_FARE[3] = {1000, 2000, 3000} paise

FUNCTION processJourney(cardBalance, zone, hour, passengerCategory)
IF zone or hour is invalid
RETURN DENIED without changing cardBalance
END IF
fare = BASE_FARE[zone]
IF (8 <= hour AND hour < 10) OR (17 <= hour AND hour < 20)
fare = fare + fare * PEAK_RATE / 100
END IF
Apply the approved concession policy, if eligible
IF cardBalance - fare < MIN_BALANCE
RETURN DENIED without changing cardBalance
END IF
cardBalance = cardBalance - fare
RETURN APPROVED and updated cardBalance
END FUNCTION
```

Both styles can produce the same fare and balance results if their logic is correct. The main difference is organisation and protection of data, not the mathematical work for one journey.

7. Complexity comparison

Measure	Procedural design	OOP design	Reason
Time per journey	O(1)	O(1)	A fixed number of zone, time, fare and balance checks is performed.

Measure	Procedural design	OOP design	Reason
Memory per card	O(1)	O(1)	Each card stores a fixed amount of information, such as an ID and balance.
Changing a fare rule	May require edits in several free functions or global-data users if duplicated.	Usually changes the central FareTable/policy and affected tests.	OOP can localise the change through responsibility and encapsulation, although dependent behaviour must still be checked.

Evaluation: Time per journey and memory per card remain O(1) in both approaches. OOP is preferred because the card balance, journey details and fare policy have distinct responsibilities. Encapsulation helps enforce the Rs 50 minimum-balance rule, while a central fare policy makes future revisions easier to manage. A procedural design may be adequate for a small program, but becomes harder to maintain as the network and fare rules grow.

8. Assumptions and policy details requiring confirmation

1. The three fares are interpreted as base fares for the farthest zone reached, as stated in the scenario.
2. Peak intervals are treated as start-inclusive and end-exclusive to resolve the stated boundary inconsistency.
3. The Rs 50 minimum is checked after all applicable fare adjustments.
4. Student/senior concession percentages and whether concession is applied before or after the peak surcharge are not specified. Confirm these before encoding them.
5. This task is an object-oriented analysis task; no execution output or program-testing claim is made.

Conclusion

The metro fare system is naturally modelled using SmartCard, Passenger, Journey, Zone and FareTable. OOP separates responsibilities and protects important rules, especially the requirement that at least Rs 50 remains after a successful journey. Although procedural and OOP solutions have the same O(1) time per journey and O(1) memory per card, OOP provides a clearer structure for extending and maintaining the fare system.

End of Task 1